

CHAPTER 10: OPERATING SYSTEMS

WHAT IS AN OPERATING SYSTEM (OS)?

- Software that **manages hardware and software resources** of a computer.
- Acts as an **interface** between the user and the hardware.
- Examples: Windows, macOS, Linux, Android.

Main goals:

- Provide a user-friendly environment.
- Allocate system resources efficiently.
- Ensure security and reliability.

OS ARCHITECTURE & STRUCTURE

A) MONOLITHIC ARCHITECTURE

- Entire OS runs in **kernel mode**.
- All services (file, process, memory, I/O) are in one large module.
- Fast but less secure — a bug can crash the whole system.
Example: early UNIX.

B) LAYERED ARCHITECTURE

- OS divided into **layers**, each built on the lower one (hardware at the bottom).
- Simplifies design and debugging.
Example: THE Operating System.

C) MICROKERNEL ARCHITECTURE

- Only essential services (CPU scheduling, memory, IPC) run in the kernel.
- Other services (drivers, file system) run in **user mode**.
- More secure and stable but slower due to extra message passing.
Example: macOS, QNX.

OS SERVICES

Service	Description	Example
Process management	Create, schedule, and terminate processes	Multitasking
Memory management	Allocate/deallocate memory, manage paging & virtual memory	Virtual memory
File management	Read/write, permissions, organization	NTFS, ext4
I/O management	Device drivers and buffering	Keyboard, printer
Security & Protection	Authentication, encryption, access control	Login, firewall
User interface	CLI/GUI for user interaction	Windows Shell, Bash

HARDWARE-OS RELATIONSHIP

- OS acts as a **bridge**:
Hardware ⇌ OS ⇌ User Applications
- Uses **device drivers** to communicate with hardware.
- Manages **CPU time**, **I/O channels**, and **memory space**.
- Converts **user requests** into **hardware actions**.

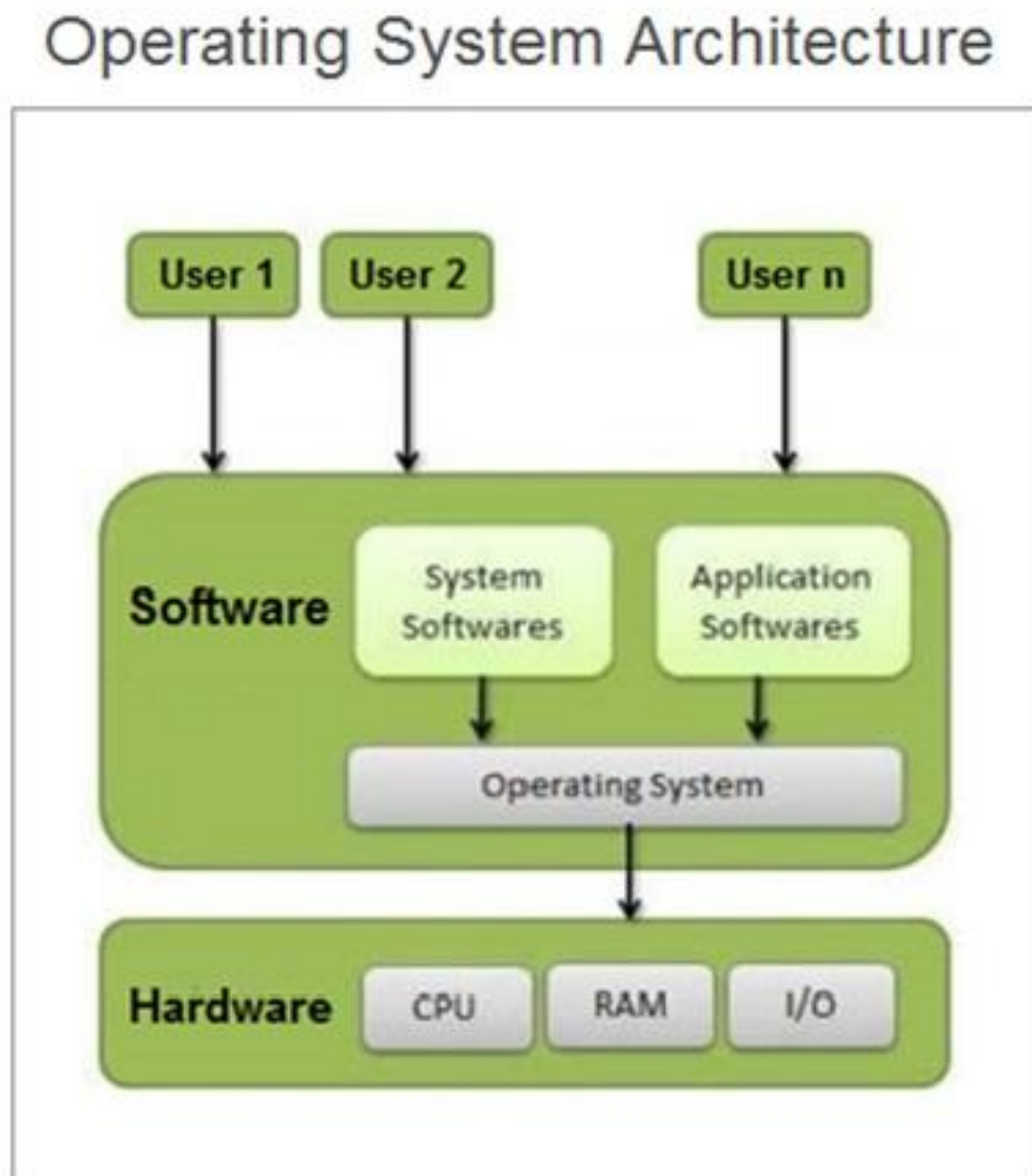
BOOTING PROCESS (STARTUP SEQUENCE)

1. **Power On Self Test (POST)** by BIOS/UEFI — checks hardware.
2. **BIOS/UEFI** finds the bootable device (HDD, SSD, USB).
3. **Boot Loader** (e.g., GRUB, Windows Boot Manager) loads OS kernel into memory.
4. **Kernel Initialization** — hardware drivers, system services, background processes.
5. **Login/GUI** presented to user.

OS CUSTOMIZATION EXAMPLES

- **Themes & interfaces:** change UI appearance (Linux desktop environments).
- **Kernel modules:** load/unload drivers or features.
- **System configuration files:** control startup behavior (Windows Registry, Linux /etc).
- **Virtualization:** customized OS instances (VMs, containers).

DIAGRAM — TYPICAL OS LAYERS



QUICK REVIEW

- Define the main role of an OS.
- List three OS services.
- Explain difference between monolithic and microkernel OS.